

Day 26: Performance Optimization & VertiPaq Engine

📌 Definition to Remember

Performance Optimization in Power BI involves reducing semantic model memory footprint and tuning DAX queries to ensure visual rendering completes in **under 1-2 seconds**.

1. Using the Performance Analyzer

- Built-in diagnostic tool (View → Performance Analyzer) recording 3 visual cost components:
 - DAX Query Time:** Time taken by the VertiPaq engine to evaluate calculations.
 - Visual Display Time:** Time taken by the frontend browser to render charts.
 - Other (Queue Wait):** Background thread concurrency delays.

2. Column Cardinality & VertiPaq Compression

- VertiPaq stores data **column-by-column** using Dictionary, Run-Length (RLE), and Bit-Packed compression.
- Cardinality (Number of Unique Values):** High cardinality columns (e.g., timestamps with milliseconds, GUID transaction keys) destroy compression and bloat model RAM.
- Optimization:** Split `DateTime` into separate `Date` and `Time` columns, or remove high-cardinality surrogate keys from the fact table.

3. DAX Optimization Best Practices

- Avoid iterating full tables with `FILTER(AllTable, ...)` when simple column filters suffice.
- Store repeated calculations in `VAR` to prevent double evaluations.
- Use `COUNTROWS(Table)` instead of `COUNT(Table[Column])`.

★ Must-Master Skills & Interview Points

- How to diagnose visual rendering lag with Performance Analyzer.
- What column cardinality is and why reducing it drastically shrinks `.pbix` file size.
- Using DAX Studio and Tabular Editor for advanced query plan and VertiPaq metrics analysis.

⚡ Quick Recall

Performance Analyzer + Reduce Column Cardinality + Split DateTimes + Cache VARs in DAX